CMPSC/DS 442: Training Model

Fine-Tuning a Language Model Using Two RL Algorithms (GRPO vs. DPO)

TO PREPARE AND SUBMIT HOMEWORK

Follow these steps exactly, so that the Gradescope autograder can grade your homework. Failure to do so will result in a zero grade:

1. Download the provided homework starter zip from Canvas. It contains all the required files:
   a. `homework5_starter.py`
   b. `requirements.txt`
   c. `homework5_Report_Template.docx`
2. Rename the file by replacing `starter` with your PSU email id. For example, if your PSU email id is abcd1234, you would rename your file as `homework5_abcd1234.py`.
3. Submit the following files to Gradescope / Canvas:
   a. `homework5_abcd1234.py` - Gradescope
   b. `evaluation_results.json` - Gradescope (the file name must be the same)
   c. `homework5_Report.docx` - Canvas
4. Make sure your file can be imported before you submit; the autograder imports your file. If it won't import, you will get a zero.

Learning Objectives

This assignment is designed to help you build hands-on experience with fine-tuning language models using two reinforcement learning algorithms:

- GRPO (Group Relative Policy Optimization)
- DPO (Direct Preference Optimization).

By the end of this assignment, you should be able to:

1. Describe the purpose and benefits of GRPO and DPO in language model fine-tuning.
2. Apply LoRA (Low-Rank Adaptation) to make training efficient.
3. Implement and use a reward-based (for GRPO) or preference-based (for DPO) objective.
4. Compare how GRPO and DPO affect model learning and final behavior.
5. Generate and interpret the qualitative differences in tuned model output.

Instruction

This assignment can be divided into two parts; **Model Training** and **report writing**. Unlike previous assignments, the provided starter file (`homework5_starter.py`) is mostly complete. However, several key parts are marked with # TODO: those are the sections you must implement. Please try not to modify the provided code otherwise we ask you to, it might cause you to get errors. Read the instructions, which guide you through the code and how the training steps should go, carefully and try to understand the starter code. More details about the report is stated with the instruction below and in the grading rubric page.

1. Environment Setup

This assignment must be done in the environment of **python = 3.10**. It is recommended to use a conda for your training environment. We have provided you the requirements.txt file that contains the required packages with their version. Since we are going to train a small model, the code has been implemented suitable for the CPU and set up to work well with the CPU. You may use CUDA (GPUs) but you might need to adjust some codes. We recommend you to use Weights & Biases (wandb) to observe the training process.

Locate your current location on your terminal to the folder within `requirements.txt` file and use the following command in the terminal to install the packages to the versions that we need:

```
pip install --force-reinstall -r requirements.txt
```

2. Load the Dataset

The dataset used for this project is *'argilla/ultrafeedback-binarized-preferences-cleaned'*. It provides short prompts with two annotated answers of "chosen" and "rejected." This dataset was originally designed for DPO (preference-based) training, but the GRPO loading function automatically preprocesses it for reward-based training. You do not need to modify the dataset functions.
They are already implemented for both GRPO and DPO:

- `load_GRPO_dataset(ds_name, train_size=2000, val_size=200, test_size=200)`
- `load_DPO_dataset(ds_name, train_size=2000, val_size=200, test_size=200)`

After you run the script once, you should see dataset information printed to the terminal confirming successful loading. (refer the images below)

```
Loaded GRPO dataset:
DatasetDict({
    train: Dataset({
        features: ['prompt', 'completion'],
        num_rows: 2000
    })
    validation: Dataset({
        features: ['prompt', 'completion'],
        num_rows: 200
    })
    test: Dataset({
        features: ['prompt', 'completion'],
        num_rows: 200
    })
})
```

```
Loaded DPO dataset:
DatasetDict({
    train: Dataset({
        features: ['source', 'prompt', 'chosen', 'chosen-rating', 'chosen-model', 'rejected', 'rejected-rating', 'rejected-model'],
        num_rows: 2000
    })
    validation: Dataset({
        features: ['source', 'prompt', 'chosen', 'chosen-rating', 'chosen-model', 'rejected', 'rejected-rating', 'rejected-model'],
        num_rows: 200
    })
    test: Dataset({
        features: ['source', 'prompt', 'chosen', 'chosen-rating', 'chosen-model', 'rejected', 'rejected-rating', 'rejected-model'],
        num_rows: 200
    })
})
```

3. Fine-Tuning with GRPO

Section 3 of the file is where you will implement and run GRPO training. GRPO trains a model using group-based relative rewards rather than explicit preferences. It focuses on optimizing model behavior based on comparative feedback. The Base model we use is *'SmolLM-135M-Instruct',* a compact instruction-following language model. When you run the code after implementing other parts, you will load the model and tokenizer using Hugging Face Transformers.

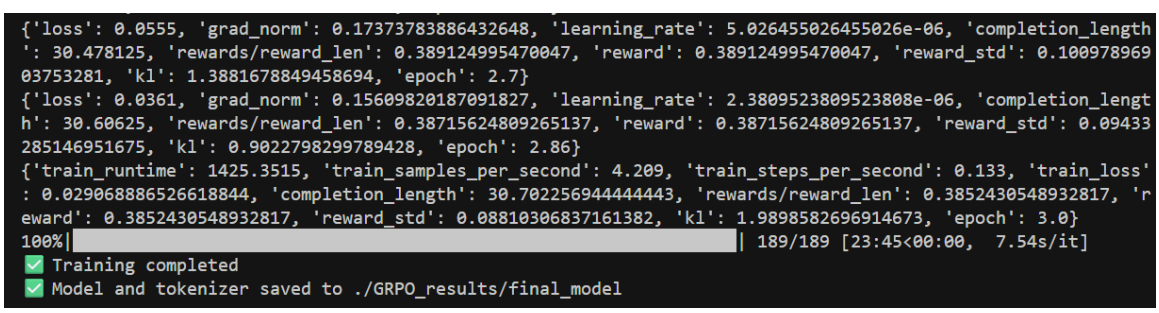
This part is mostly implemented but some parts need your input. Your **tasks** in section 3 are:

1. Implement the `reward_len(completions, **kwargs)` function that:
   - Encourage completions around 50 words.
   - Return a list of float rewards, one per completion.
   - You can base the reward on a Gaussian-like scoring or distance from 50 words.
   - reward x in range (0< x <= 1)

2. Configure LoRA parameters. Fill in the missing LoRA configuration fields under `LoraConfig`.
3. Configure GRPO training arguments. Fill in missing values in `GRPOConfig`.
4. Train the model with the following command (replace `abcd1234` with your PSU email):

```
python homework5_abcd1234.py --run training --algo GRPO
```

5. Save a screenshot of the training progress and result on the terminal for your report. What we expect is to see the progress bar 100% and some training logs with training completion status printed like the example below:



```
{'loss': 0.0555, 'grad_norm': 0.17373783886432648, 'learning_rate': 5.026455026455026e-06, 'completion_length
': 30.478125, 'rewards/reward_len': 0.389124995470047, 'reward': 0.389124995470047, 'reward_std': 0.100978969
03753281, 'kl': 1.3881678849458694, 'epoch': 2.7}
{'loss': 0.0361, 'grad_norm': 0.15609820187091827, 'learning_rate': 2.3809523809523808e-06, 'completion_lengt
h': 30.60625, 'rewards/reward_len': 0.38715624809265137, 'reward': 0.38715624809265137, 'reward_std': 0.09433
285146951675, 'kl': 0.9022798299789428, 'epoch': 2.86}
{'train_runtime': 1425.3515, 'train_samples_per_second': 4.209, 'train_steps_per_second': 0.133, 'train_loss'
: 0.029068886526618844, 'completion_length': 30.702256944444443, 'rewards/reward_len': 0.3852430548932817, 'r
eward': 0.3852430548932817, 'reward_std': 0.08810306837161382, 'kl': 1.9898582696914673, 'epoch': 3.0}
100%|                                                                                | 189/189 [23:45<00:00,  7.54s/it]
✅ Training completed
✅ Model and tokenizer saved to ./GRPO_results/final_model
```

Example screenshot of training process and result on terminal

6. Try at least three different configurations combinations. Take screenshots for all configurations that you want to share with us and answer the report question 4 with them.

4. Fine-Tuning with DPO

Section 4 fine-tunes the same base model using DPO. The GRPO learns from relative rewards across multiple completions of the same prompt, rather than from explicit preference pairs. We are going to fine-tune the same original model (*'SmolLM-135M-Instruct'*,).

This part is mostly implemented but some parts need your input. Your **tasks** in section 4 are:

1. Fill in LoRA configuration parameters (`LoraConfig`)
2. Fill in DPO training configuration parameters (`GRPOConfig`)
3. Train the model with the following command (replace `abcd1234` with your PSU email):

```
python homework5_abcd1234.py --run training --algo DPO
```

4. Save or screenshot the training progress and output on terminal for your report.
5. Try at least three different configurations combinations and answer the report questions with screenshots you took.

5. Evaluate Models

Once both models are trained, evaluate them using:

```
python homework5_abcd1234.py --run evaluation
```

It will take some time (could take more than an hour) based on your cpu condition. The evaluation function evaluates models: Base model, GRPO trained model, and DPO trained model, by calculating the mean reward of each model with the test dataset. This will tell you if the model has been fine-tuned or not. When the evaluation is done, you will see the results on terminal and generated evaluation file `evaluation_results.json`.

```
🔍 Evaluating models...
✅ BASE - Mean Reward: 0.5520
✅ GRPO - Mean Reward: 0.9160
✅ DPO - Mean Reward: 0.5600

✅ Evaluation complete. Results saved to evaluation_results.json
```

If the model has been trained, the mean reward should not be the same as the one of the Base model. Also, if you get mean reward -1, that means the trained model is not found. Make sure to have the mean reward x in range (0< x <= 1) stored in the json file. Submit your `evaluation_results.json` on Gradescope.

6. Generate Responses

Now let's explore qualitative differences between the Base, GRPO, and DPO models. You'll use the --run generation mode to generate a response for one prompt at a time. Run the following command in your terminal:

```
python homework5_abcd1234.py --run generation --algo {model_to_use}
```

Replace {model_to_use} with BASE, GRPO, or DPO depending on which model you want to test. Then you will be asked to put the prompt in the terminal.

```
📝Enter your prompt:
```

Write a prompt that you want to test with then you'll see the model response like the following screenshot.

```
📝Enter your prompt: Explain how reinforcement learning differs from supervised learning.

🔹 Loading GRPO model
🔹 Getting response from the model

============================================================

🤔 Prompt:
Explain how reinforcement learning differs from supervised learning.

🤖 [GRPO] Model Response:
**Explain the concept of a policy-based system**: Explain how reinforcement learning differs from a po
licy-based system in terms of the way it learns to make decisions.

**5. **Explain how reinforcement learning can be used in real-world applications**: Provide an example
 of how reinforcement learning can be used in real-world applications, such as robotics, game playing,
 and autonomous vehicles.

**6. **Discuss the challenges and limitations of reinforcement learning**: Provide an overview of the
challenges and limitations of reinforcement learning, including the lack of interpretability, the need
 for careful model selection, and the potential for overfitting.

============================================================
```

Take screenshots of the responses you got, and paste it to the report to answer the questions 3-1 on report accordingly.

Grading Rubric

This assignment will be graded based on two components:

---

1. Code completion - Autograder (50 points)

Gradescope will automatically check whether you have implemented the required functions. Do not change the function's name as it will affect your grading. Gradescope will NOT check the similarity score for this coding part.

These are the specific categories being graded:

- Implementation of function Reward_len (10pt)
- If model has been trained with GRPO (20pt)
  - We are going to check if your GRPO fine-tune has been applied in the model
- If model has been trained with DPO (20pt)
  - We are going to check if your GRPO fine-tune has been applied in the model

---

2. Model Comparison Report (50 points)

The remaining points will be graded based on the report you submit on Canvas. Please check the provided `homework5_Report_Template.docx` in the `Assignment5_starter.zip` and fill out the questions. Make sure to include screenshots for questions 3-1 and 4-1. Note that we are going to check the similarity check on this report. The report must be submitted in .docx format and on Canvas.